

Universidad Nacional Autónoma de México

Introducción a Redes Neuronales

Implementación de un modelo Transformer para la detección de correos electrónicos maliciosos

Ballesteros Luna Nathalia Jazmín	319130425
Macías Moto Luis Armando	315056015

Junio de 2026

Índice

1. Introducción	2
2. Datos: Muestreo y Data Augmentation	2
2.1. Recolección y Muestreo de los Datos	2
2.2. Uso de Data Augmentation	2
3. Topología de la Red	3
3.1. Diseño de la Arquitectura	3
3.2. Por qué no usamos Transfer Learning	5
4. Configuración del Entrenamiento	5
4.1. Métricas y Función de Pérdida	5
4.2. Uso de Callbacks	5
5. Prevención del Sobreajuste (Overfitting)	5
6. Conclusiones y Resultados	6
6.1. Análisis de Falsos Positivos y Sesgos de Vocabulario	6
6.2. Pruebas en la Interfaz Gráfica	7

1. Introducción

El objetivo de este proyecto es construir un modelo de aprendizaje profundo que pueda leer textos y correos electrónicos para clasificarlos en tres categorías distintas: contenido seguro (0), intentos de phishing (1) y ataques de inyección SQL (2).

Identificar estas tres categorías al mismo tiempo es útil porque los ataques modernos suelen llegar ocultos en textos de apariencia normal. Para lograrlo, se diseñó una red neuronal que procesa tanto el texto escrito como características específicas extraídas de forma manual.

2. Datos: Muestreo y Data Augmentation

2.1. Recolección y Muestreo de los Datos

Para entrenar el modelo, juntamos información de varias bases de datos públicas, como Enron y Ling para correos seguros, y Nazario y CEAS para correos de phishing, además de un conjunto específico para inyecciones SQL.

Al unir todo, notamos que había muchas más muestras de algunas categorías que de otras. Si dejábamos los datos así, el modelo se iba a acostumbrar a predecir siempre la clase con más ejemplos. Para evitar esto, usamos una técnica de submuestreo (under-sampling) con la herramienta `resample` de scikit-learn. Lo que hicimos fue limitar la cantidad de ejemplos de todas las clases al número que tenía la clase más pequeña. Al final, nos quedamos con 11,340 muestras por cada categoría, logrando un conjunto de datos equilibrado.

```
1 counts = df_raw['Target'].value_counts()
2 min_count = counts.min()
3 target_n = min(min_count, 15000)
4
5 balanced_parts = []
6 for cls in df_raw['Target'].unique():
7     cls_df = df_raw[df_raw['Target'] == cls]
8     if len(cls_df) > target_n:
9         cls_df = resample(cls_df, n_samples=target_n, replace=False,
10                           random_state=42)
11     balanced_parts.append(cls_df)
12 df_balanced = pd.concat(balanced_parts, ignore_index=True)
```

Listing 1: Lógica de balanceo de clases en `data_loader.py`

2.2. Uso de Data Augmentation

Durante las pruebas, encontramos un problema con los idiomas. La mayoría de los correos de phishing en nuestras bases de datos estaban en inglés, mientras que agregamos varios ejemplos de correos seguros en español para darle más variedad. El modelo aprendió equivocadamente que cualquier texto en español era seguro.

Para solucionar este sesgo, decidimos usar *Data Augmentation*. Escribimos y agregamos manualmente ejemplos de phishing en español (con y sin enlaces). Esto ayudó a que el modelo entendiera que las palabras maliciosas también existen en español, corrigiendo el problema de forma efectiva.

3. Topología de la Red

3.1. Diseño de la Arquitectura

Decidimos usar una red de doble entrada. Esto significa que el modelo recibe la información por dos caminos distintos antes de tomar una decisión:

- **Primera entrada:** Recibe el texto directo (después de limpiarlo) y lo pasa por un bloque Transformer para entender el contexto de las palabras.
- **Segunda entrada:** Recibe 14 características numéricas que calculamos previamente (como cuántas URLs sospechosas hay o si contiene comandos de bases de datos).

Al principio, usábamos una capa llamada `GlobalAveragePooling1D` para resumir la información del texto. Sin embargo, nos dimos cuenta de que si un ataque de SQL era muy corto y el correo muy largo, el promedio diluía el valor del ataque y el modelo no lo detectaba. Lo cambiamos por `GlobalMaxPooling1D`, que en lugar de sacar un promedio, se queda con el valor más alto. Esto permitió que las señales de ataque resaltaran mejor.

```
1 # Rama de NLP (Transformer)
2 text_input = Input(shape=(sequence_length,), name="text_input")
3 embedding = layers.Embedding(input_dim=max_tokens, output_dim=64)(
4     text_input)
5 # ... (Capas de Atencion y Normalizacion) ...
6 text_vec = layers.GlobalMaxPooling1D()(seq_out)
7 text_dense = layers.Dense(16, activation="relu")(text_vec)
8
9 # Rama de características manuales
10 feat_input = Input(shape=(N_FEATURES,), name="feat_input")
11 feat_dense = layers.Dense(32, activation="relu")(feat_input)
12 feat_dense = layers.BatchNormalization()(feat_dense)
13 feat_out = layers.Dense(16, activation="relu")(feat_dense)
14
15 # Concatenacion final
16 concat = layers.Concatenate()([text_dense, feat_out])
17 outputs = layers.Dense(3, activation="softmax")(concat)
```

Listing 2: Definición de la arquitectura Dual-Input en `trainer.py`

Layer (type)	Output Shape	Param #	Connected to
text_input (InputLayer)	(None, 256)	0	-
embedding (Embedding)	(None, 256, 64)	512,000	text_input[0][0]
multi_head_attention (MultiHeadAttention)	(None, 256, 64)	16,640	embedding[0][0], embedding[0][0]
dropout_1 (Dropout)	(None, 256, 64)	0	multi_head_attention[0][0]
add (Add)	(None, 256, 64)	0	embedding[0][0], dropout_1[0][0]
layer_normalization (LayerNormalization)	(None, 256, 64)	128	add[0][0]
dense (Dense)	(None, 256, 64)	4,160	layer_normalization[0][0]
dense_1 (Dense)	(None, 256, 64)	4,160	dense[0][0]
dropout_2 (Dropout)	(None, 256, 64)	0	dense_1[0][0]
add_1 (Add)	(None, 256, 64)	0	layer_normalization[0][0], dropout_2[0][0]
feat_input (InputLayer)	(None, 14)	0	-
layer_normalization_1 (LayerNormalization)	(None, 256, 64)	128	add_1[0][0]
dense_3 (Dense)	(None, 32)	480	feat_input[0][0]
global_max_pooling1d (GlobalMaxPooling1D)	(None, 64)	0	layer_normalization_1[0][...]
batch_normalization (BatchNormalization)	(None, 32)	128	dense_3[0][0]
dense_2 (Dense)	(None, 16)	1,040	global_max_pooling1d[0][0]
dense_4 (Dense)	(None, 16)	528	batch_normalization[0][0]
concatenate (Concatenate)	(None, 32)	0	dense_2[0][0], dense_4[0][0]
dropout_3 (Dropout)	(None, 32)	0	concatenate[0][0]
dense_5 (Dense)	(None, 64)	2,112	dropout_3[0][0]
dropout_4 (Dropout)	(None, 64)	0	dense_5[0][0]
dense_6 (Dense)	(None, 3)	195	dropout_4[0][0]
Total params: 541,699 (2.07 MB)			
Trainable params: 541,635 (2.07 MB)			
Non-trainable params: 64 (256.00 B)			
Class weights: {0: 1.0, 1: 1.0, 2: 1.0}			

Figura 1: Resumen de las capas de la arquitectura del modelo.

3.2. Por qué no usamos Transfer Learning

Consideramos usar modelos ya preentrenados grandes como BERT (*Transfer Learning*), pero decidimos no hacerlo. La razón principal es que este proyecto requiere detectar código SQL (como comillas simples, asteriscos o comandos de bases de datos), y los modelos generales suelen ignorar estos caracteres porque están entrenados para lenguaje humano normal. Entrenar nuestra propia capa de palabras (*Embedding*) desde cero fue más ligero para la computadora y más adecuado para este tipo de problema.

4. Configuración del Entrenamiento

4.1. Métricas y Función de Pérdida

Como nuestro modelo tiene que elegir solo una opción entre tres posibles categorías, utilizamos la función de pérdida `sparse_categorical_crossentropy`. Para medir qué tan bien estaba aprendiendo, observamos la exactitud general (`sparse_categorical_accuracy`) en cada ciclo.

Para el optimizador, elegimos **Adam** con una tasa de aprendizaje (learning rate) inicial de 0,001. Adam es muy útil porque ajusta la velocidad de aprendizaje de forma automática, lo que ayuda a que el modelo no tarde demasiado en entrenar.

4.2. Uso de Callbacks

Para evitar tener que vigilar el entrenamiento manualmente, usamos tres herramientas de Keras llamadas callbacks:

- **EarlyStopping:** Sirve para detener el proceso si el modelo deja de mejorar después de 3 intentos. Así ahorramos tiempo y recursos.
- **ReduceLROnPlateau:** Si el modelo se estanca, esta herramienta baja la tasa de aprendizaje para que los ajustes en la red sean más pequeños y precisos.
- **ModelCheckpoint:** Va guardando el modelo únicamente cuando obtiene un mejor resultado que el anterior.

```
1 callbacks = [  
2     EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=  
3     True),  
4     ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=2, min_lr=  
5     1e-5),  
6     ModelCheckpoint(keras_model_path, save_best_only=True, monitor='  
7     val_loss')  
8 ]
```

Listing 3: Configuración de Callbacks

5. Prevención del Sobreajuste (Overfitting)

Un problema común al entrenar redes neuronales es que el modelo memorice los datos en lugar de aprender los patrones reales (sobreajuste). En nuestras primeras pruebas notamos que esto estaba ocurriendo, así que aplicamos un par de técnicas para reducirlo:

Primero, agregamos capas de **Dropout** (con valores de 0.1 y 0.2). Estas capas apagan partes de la red neuronal de forma aleatoria durante el entrenamiento, obligando al modelo a usar caminos diferentes y no depender de un solo patrón.

Segundo, usamos **LayerNormalization** y **BatchNormalization**. Esto sirve para mantener los valores numéricos controlados y en una misma escala a medida que pasan por las distintas capas de la red, lo que le da más estabilidad al aprendizaje. Además, aplicamos pesos a las clases durante el entrenamiento (*class weights*) para que el modelo le diera la misma importancia a todas las categorías.

6. Conclusiones y Resultados

El trabajo demostró que separar el texto de las características manuales en dos entradas distintas es una forma funcional de abordar este problema de clasificación. Al balancear los datos correctamente y aplicar las capas de regularización, el modelo fue capaz de clasificar los datos de prueba con una buena precisión. Las decisiones sobre la arquitectura, especialmente el uso del Max Pooling, fueron clave para identificar las inyecciones SQL de forma correcta.

```

--- Training ---
Epoch 1/25
745/745 98s 121ms/step - loss: 0.1339 - sparse_categorical_accuracy: 0.9471 - val_loss: 0.0235 - val_sparse_categorical_accuracy: 0.9912 - learning_rate: 0.0010
Epoch 2/25
745/745 91s 122ms/step - loss: 0.0214 - sparse_categorical_accuracy: 0.9936 - val_loss: 0.0213 - val_sparse_categorical_accuracy: 0.9924 - learning_rate: 0.0010
Epoch 3/25
745/745 89s 120ms/step - loss: 0.0121 - sparse_categorical_accuracy: 0.9966 - val_loss: 0.0266 - val_sparse_categorical_accuracy: 0.9927 - learning_rate: 0.0010
Epoch 4/25
745/745 91s 122ms/step - loss: 0.0092 - sparse_categorical_accuracy: 0.9973 - val_loss: 0.0260 - val_sparse_categorical_accuracy: 0.9935 - learning_rate: 0.0010
Epoch 5/25
745/745 92s 123ms/step - loss: 0.0045 - sparse_categorical_accuracy: 0.9984 - val_loss: 0.0272 - val_sparse_categorical_accuracy: 0.9939 - learning_rate: 2.0000e-04

--- Evaluating on Test Set ---
160/160 6s 35ms/step - loss: 0.0241 - sparse_categorical_accuracy: 0.9916
Test Loss: 0.0241 | Test Accuracy: 0.9916
160/160 6s 38ms/step

--- Confusion Matrix ---
[[1689  1  1]
 [ 24 1673  4]
 [ 2  1 1698]]

--- Classification Report ---

```

	precision	recall	f1-score	support
Safe (0)	0.98	0.99	0.99	1701
Phishing (1)	0.99	0.98	0.99	1701
SQLi (2)	1.00	1.00	1.00	1701
accuracy			0.99	5103
macro avg	0.99	0.99	0.99	5103
weighted avg	0.99	0.99	0.99	5103

Figura 2: Matriz de confusión y reporte obtenidos al finalizar el entrenamiento.

6.1. Análisis de Falsos Positivos y Sesgos de Vocabulario

Durante las pruebas de inferencia, identificamos un comportamiento predecible en la red neuronal. Al intentar corregir el sesgo de idioma (el cual asumía que el español era seguro), inyectamos una cantidad considerable de correos de phishing sintéticos. Esto provocó que el modelo desarrollara una fuerte sensibilidad a palabras como “urgente”, “password” o “account”.

Como resultado, observamos que si un ataque de inyección SQL se camufla dentro de un texto que abusa de estas palabras de ingeniería social, el modelo a veces se inclina por clasificarlo como Phishing. Esto se debe a que el peso acumulado de las palabras sospechosas en la capa de atención supera el valor numérico del ataque SQL en la capa de MaxPooling. Es un hallazgo valioso que nos ayuda a entender cómo toma decisiones la red y nos indica áreas de mejora para futuros entrenamientos.

6.2. Pruebas en la Interfaz Gráfica

Para validar el funcionamiento del modelo en tiempo real, desarrollamos una interfaz de usuario. A continuación, se presentan las pruebas de inferencia que demuestran la capacidad del modelo para clasificar distintos tipos de correos tanto en inglés como en español.

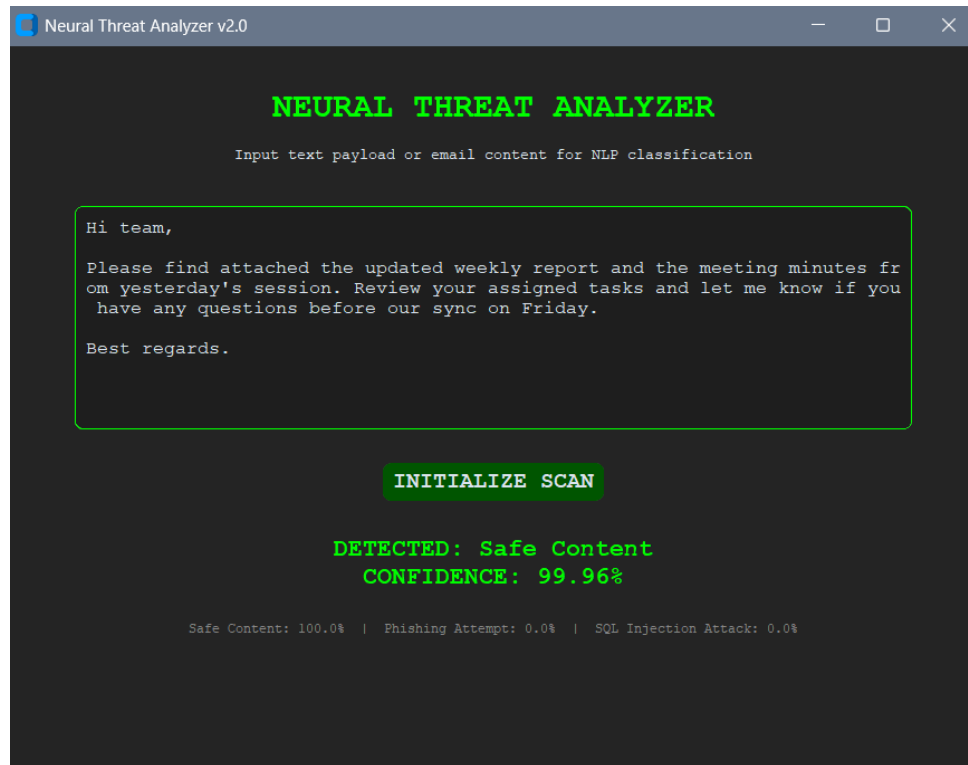


Figura 3: Detección exitosa de un correo corporativo legítimo en inglés (Clase 0).

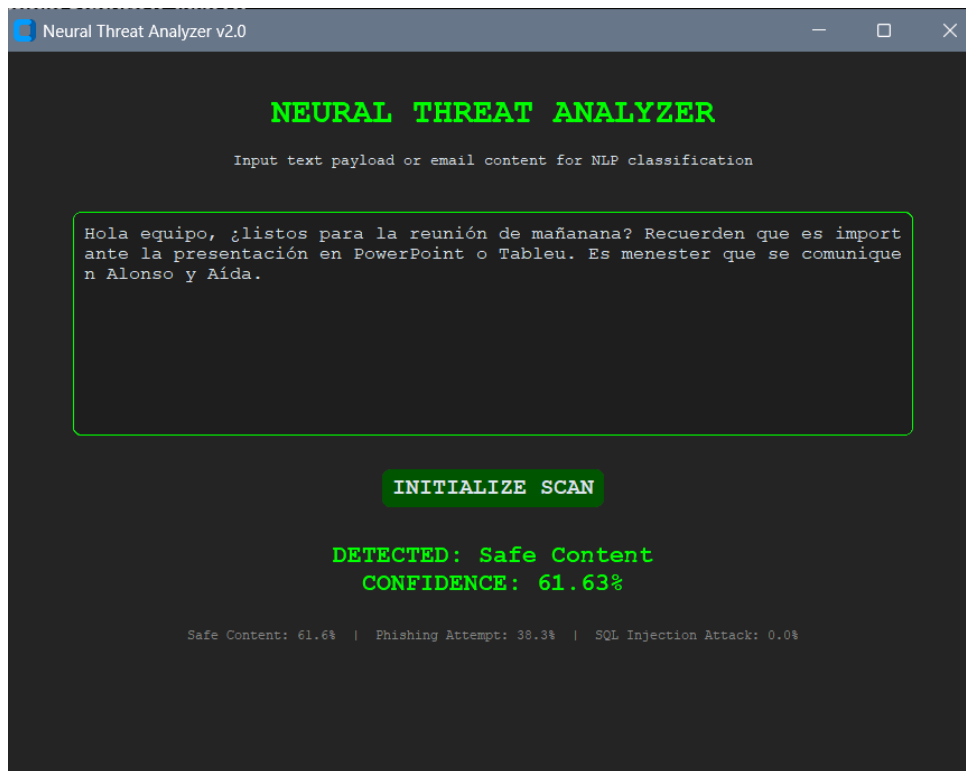


Figura 4: Detección exitosa de un correo corporativo legítimo en español (Clase 0).

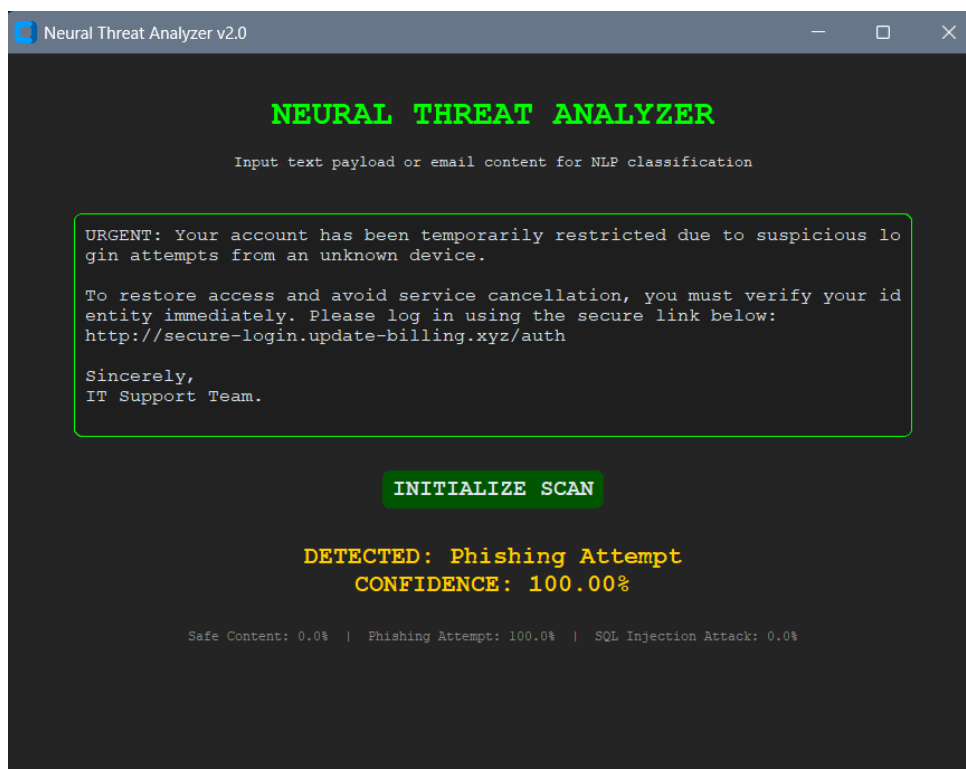


Figura 5: Identificación de un intento de Phishing por ingeniería social y enlaces sospechosos (Clase 1).

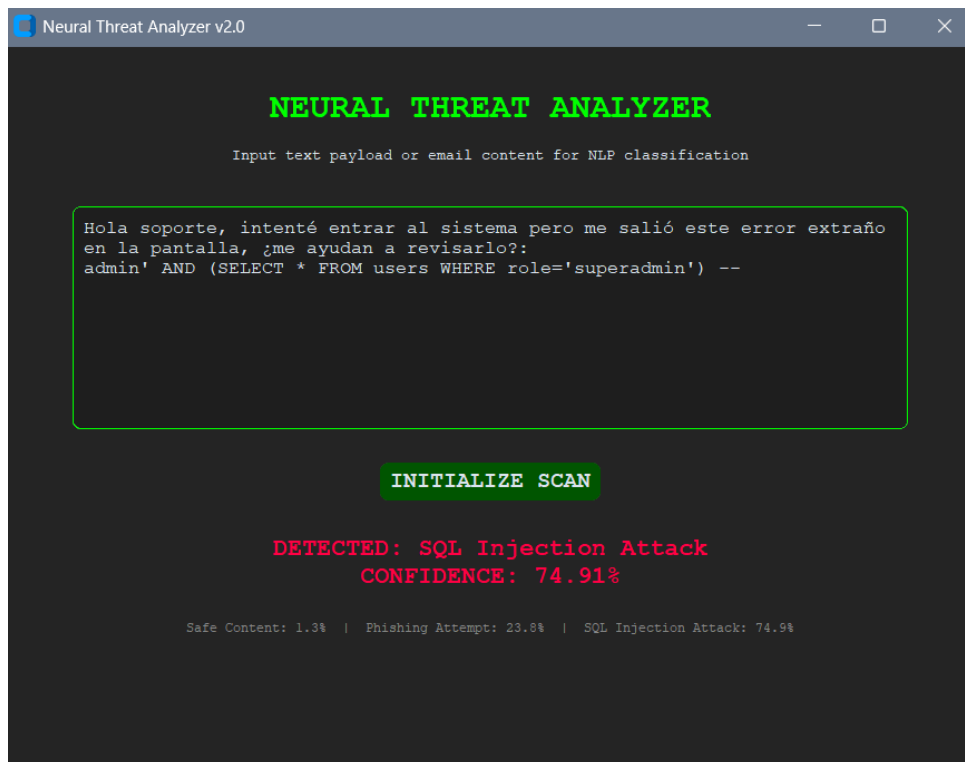


Figura 6: Detección de una Inyección SQL ofuscada en una solicitud de soporte técnico (Clase 2).

```

--- CLEAN TEXT ---
hola equipo listos para la reunion de mananana recuerden que es importante la presentacion en powerpoint o tableu es urgente que se comuniquen alonso y aida
--- FEATURES ---
[0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0.8695652173913043, 0]
--- CLEAN TEXT ---
hola equipo listos para la reunion de mananana recuerden que es importante la presentacion en powerpoint o tableu es menester que se comuniquen alonso y aida
--- FEATURES ---
[0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0.0, 0]
--- CLEAN TEXT ---
hi team please find attach update weekly report and meeting minute from yesterday session review your assign task and let know question sync friday good regard
--- FEATURES ---
[0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0.0, 0]
--- CLEAN TEXT ---
urgent your account temporarily restrict suspicious login attempt from unknown device restore access and avoid service cancellation must verify your identity immediately please log secure link url_suspicious_tld sincerely support team
--- FEATURES ---
[0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0.0, 0]
--- CLEAN TEXT ---
hello support try reset password system keep give database error type username enter admin sql_tautology sql_comment please check wrong account
--- FEATURES ---
[0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0.0, 0]
--- CLEAN TEXT ---
hello support try reset password system keep give database error type username enter admin sql_tautology sql_comment
--- FEATURES ---
[0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0.0, 0]
--- CLEAN TEXT ---
hello support try reset password system keep give database error admin sql_tautology sql_comment
--- FEATURES ---
[0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0.0, 0]
--- CLEAN TEXT ---
hello support try reset password system keep give admin sql_tautology sql_comment
--- FEATURES ---
[0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0.0, 0]
--- CLEAN TEXT ---
hello support try reset password system keep give admin sql_tautology sql_comment
--- FEATURES ---
[0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0.0, 0]

```

Figura 7: Salida del sistema backend mostrando el preprocesamiento de texto y la extracción de vectores de características de las inferencias anteriores.